

TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

REPORT ON LABORATORY WORK NO. 9

OPERATING SYSTEMS

Watching System Calls with strace

Done by: st. gr. FAF-241, Lucian-Adrian Gavril

Checked by: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

INTRODUCTION 3
1 THE SYSTEM CALLS OF A REAL COMMAND 4
1.1 strace on ls -l 4
1.2 How many calls 4
2 A PROGRAM THAT CALLS THE KERNEL DIRECTLY 6
2.1 Build and run 6
2.2 The same program seen by strace 6
3 COUNTING ONE KIND OF CALL 8
3.1 The whole profile of syscalls 8
3.2 write across programs 8
3.3 Buffering decides the count 9
CONCLUSIONS 10

INTRODUCTION

Purpose of the work. To watch the boundary between a program and the kernel. `strace` prints every system call a process makes, so the work reads the calls of a real command (`ls -l`), writes a program that uses `open`, `write`, `read` and `close` directly and checks that each C call becomes exactly one system call, and counts one kind of call across several programs to see what decides how many times a program enters the kernel.

Environment. Debian GNU/Linux 13, Linux kernel 6.12, GCC 14, strace 6.13. All commands ran in an ordinary user shell. Every terminal figure is the real output of the commands shown in it; the raw traces are kept in `evidence/`.

1 THE SYSTEM CALLS OF A REAL COMMAND

1.1 strace on ls -l

`strace -f -e trace=openat,read,write,close ls -l` prints the four selected calls beside the normal output (Figure 1.1). The first part is the dynamic loader and the C library: `openat` on `ld.so.cache` and on each shared library, each library followed by one `read` of its ELF header and a `close`. Then `ls` opens the locale files, `/etc/nsswitch.conf`, `/etc/passwd` and `/etc/group` to turn the numeric owner and group into names for the long listing. The listing itself leaves the process in a single call at the end.

```
dev1@dev1:~/os-lab9$ cd ~/os-lab9
dev1@dev1:~/os-lab9$ strace -f -o ls.trace -e trace=openat,read,write,close ls -l
total 16
-rw-rw-r-- 1 dev1 dev1    0 Nov  2 17:43 a.txt
-rw-rw-r-- 1 dev1 dev1    0 Nov  2 17:43 b.txt
-rw-rw-r-- 1 dev1 dev1 5799 Nov  2 17:43 ls.trace
drwxrwxr-x 2 dev1 dev1 4096 Nov  2 17:43 sub
-rw-rw-r-- 1 dev1 dev1 1191 Nov  2 17:43 syscalls.c
dev1@dev1:~/os-lab9$ tail -14 ls.trace
3273705 close(4)                        = 0
3273705 close(4)                        = 0
3273705 openat(AT_FDCWD, "/etc/group", O_RDONLY|O_CLOEXEC) = 4
3273705 read(4, "root:x:0:\ndaemon:x:1:\nbin:x:2:\ns"..., 4096) = 1054
3273705 close(4)                        = 0
3273705 close(3)                        = 0
3273705 openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = 3
3273705 read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\20\0\0\0\20\0\0\0\0"..., 4096) = 2424
3273705 read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\20\0\0\0\20\0\0\0\0"..., 4096) = 1509
3273705 close(3)                        = 0
3273705 write(1, "total 16\n-rw-rw-r-- 1 dev1 dev1 "..., 250) = 250
3273705 close(1)                        = 0
3273705 close(2)                        = 0
3273705 +++ exited with 0 +++
```

Figure 1.1 – strace -f -e trace=openat,read,write,close ls -l

1.2 How many calls

`strace -c` gives the exact count instead of an estimate (Figure 1.2). With the same filter, one `ls -l` makes **87** calls: 39 `openat`, 33 `close`, 14 `read` and 1 `write`. Without any filter it makes **214** system calls in total, most of them `mmap`, `fstat`, `statx` and `getxattr` (one `statx` and the extended-attribute lookups per listed file). Part of the `openat` calls fail with `ENOENT`: these are locale paths that `ls` tries before the one that exists.

```
dev1@dev1:~/os-lab9$ wc -l < ls.trace ; grep -c -v '^[0-9]* +++' ls.trace
88
87
dev1@dev1:~/os-lab9$ sed 's/^[0-9]* //' ls.trace | grep -o '^[a-z]*' | sort | uniq -c | sort -rn
     39 openat
     33 close
     14 read
      1 write
dev1@dev1:~/os-lab9$ strace -f -o all.trace ls -l > /dev/null ; grep -vc '+++' all.trace
214
dev1@dev1:~/os-lab9$ grep write ls.trace
3273705 write(1, "total 16\n-rw-rw-r-- 1 dev1 dev1 "..., 250) = 250
```

Figure 1.2 – Exact call counts for ls -l, filtered and unfiltered

Observe 1

Which system call does ls use to print to the screen? To which descriptor does it write? `ls` prints with the `write` system call, on descriptor **1**, standard output: `write(1, "total 16\n...", 250) = 250` in Figure 1.1. The whole listing goes out in that one call, because the C library collects the lines in the `stdout` buffer and flushes them together when `ls` exits.

2 A PROGRAM THAT CALLS THE KERNEL DIRECTLY

2.1 Build and run

`syscalls.c` is the program from the brief, with the return values of `open`, `write` and `read` also checked, so a failure prints `perror` instead of continuing with a bad descriptor. It builds with `-Wall -Wextra` and no warnings, and prints exactly the expected three lines (Figure 2.1). `note.txt` holds the 14 bytes `hello, kernel\n`.

```
dev1@dev1:~/os-lab9$ gcc -Wall -Wextra -o syscalls syscalls.c && echo build ok
build ok
dev1@dev1:~/os-lab9$ ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
dev1@dev1:~/os-lab9$ cat note.txt ; ls -l note.txt
hello, kernel
-rw-r--r-- 1 dev1 dev1 14 Nov  2 17:43 note.txt
```

Figure 2.1 – Build with -Wall -Wextra and run of syscalls

2.2 The same program seen by strace

`strace -e trace=openat,write,read,close ./syscalls` shows the six calls of `main` after the loader lines (Figure 2.2). Each C call is one system call with the same arguments: `open(...)` becomes

```
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
```

because glibc implements `open` with `openat` relative to the current directory; `write(3, "hello, kernel\n", 14) = 14`; `close(3) = 0`; the second `openat` gets 3 again, because 3 was freed by the `close`; `read(3, ..., 63) = 14` asks for 63 bytes (`sizeof(buf) - 1`) and receives the 14 that exist.

The three `printf` lines do not appear as three writes. They appear at the end as one `write(1, ..., 80)`: when `stdout` is not a terminal (here strace's output is redirected), the C library buffers the whole output and writes it once at `exit`.

```
dev1@dev1:~/os-lab9$ strace -o syscalls.trace -e trace=openat,write,read,close ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
dev1@dev1:~/os-lab9$ cat syscalls.trace
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0p\236\2\0\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "hello, kernel\n", 14)         = 14
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
close(3)                                = 0
write(1, "open returned descriptor 3\nwrite"..., 80) = 80
+++ exited with 0 +++
```

Figure 2.2 – strace of syscalls: the program's own calls after the loader

Observe 2

The descriptor returned by the first openat is 3, not 0, 1 or 2. Why are those three already taken? Every process starts with three descriptors already open, inherited from the shell through `fork` and kept through `exec`: 0 is standard input, 1 standard output, 2 standard error. In a terminal all three point to the terminal device. The kernel always returns the *lowest free* descriptor, and the lowest free one is 3.

Figure 2.3 proves both halves of this. `ls -l /proc/self/fd` shows 0, 1 and 2 pointing to `/dev/pts/2`. Then `./syscalls 0<&-` starts the program with descriptor 0 closed, and `open` returns **0**: the number is not reserved for anything, it was simply in use.

```
dev1@dev1:~/os-lab9$ ls -l /proc/self/fd
total 0
lrwx------ 1 dev1 dev1 64 Nov  2 17:43 0 -> /dev/pts/2
lrwx------ 1 dev1 dev1 64 Nov  2 17:43 1 -> /dev/pts/2
lrwx------ 1 dev1 dev1 64 Nov  2 17:43 2 -> /dev/pts/2
lr-x------ 1 dev1 dev1 64 Nov  2 17:43 3 -> /proc/3273735/fd
dev1@dev1:~/os-lab9$ ./syscalls 0<&- ; echo "exit: $?"
open returned descriptor 0
write sent 14 bytes
read got 14 bytes: hello, kernel
exit: 0
```

Figure 2.3 – Descriptors 0, 1, 2 of a process, and open returning 0 when stdin is closed

3 COUNTING ONE KIND OF CALL

3.1 The whole profile of syscalls

`strace -c ./syscalls` counts every call the program makes (Figure 3.1): **41** in total. Only six belong to `main`; the rest is the work of starting a C program: `execve`, 3 `brk`, 8 `mmap`, 3 `mprotect`, `openat` and `read` of `libc.so.6`, `arch_prctl`, `set_tid_address`, `rseq` and others. The counts match the program exactly: 2 `write` (one to `note.txt`, one flush of `stdout`), 2 `read` (`libc` header and `note.txt`), 4 `openat` and 4 `close`.

```
dev1@dev1:~/os-lab9$ strace -c -o count.txt ./syscalls > /dev/null ; cat count.txt
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         2           read
  0.00    0.000000           0         2           write
  0.00    0.000000           0         4           close
  0.00    0.000000           0         3           fstat
  0.00    0.000000           0         8           mmap
  0.00    0.000000           0         3           mprotect
  0.00    0.000000           0         1           munmap
  0.00    0.000000           0         3           brk
  0.00    0.000000           0         1         1 ioctl
  0.00    0.000000           0         2           pread64
  0.00    0.000000           0         1         1 access
  0.00    0.000000           0         1           execve
  0.00    0.000000           0         1           arch_prctl
  0.00    0.000000           0         1           set_tid_address
  0.00    0.000000           0         4           openat
  0.00    0.000000           0         1           set_robust_list
  0.00    0.000000           0         1           prlimit64
  0.00    0.000000           0         1           getrandom
  0.00    0.000000           0         1           rseq
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000           0        41         2 total
```

Figure 3.1 – strace -c ./syscalls: 41 system calls

3.2 write across programs

`strace -c -e trace=write` counts only `write` (Figure 3.2). `ls -l /` makes 1; `seq 1 100000` makes 72 to send 588895 bytes, that is one call per full 8 KiB buffer (about 8192 bytes each); a shell loop of five `echo` commands makes 5, one per `echo`, because `echo` is a shell built-in that writes its line immediately. The number of `write` calls depends on how the program buffers, not on how many lines it prints.

```
dev1@dev1:~/os-lab9$ strace -c -e trace=write -o w1.txt ls -l / > /dev/null ; tail -3 w1.txt
  0.00    0.000000           0         1           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000           0         1           total
dev1@dev1:~/os-lab9$ strace -c -e trace=write -o w2.txt seq 1 100000 > /dev/null ; tail -3 w2.txt ; seq
1 100000 | wc -c
  0.00    0.000000           0        72           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000           0        72           total
588895
dev1@dev1:~/os-lab9$ strace -c -e trace=write -o w3.txt sh -c 'for i in 1 2 3 4 5; do echo $i; done' > /
dev/null ; tail -3 w3.txt
100.00    0.000006           1         5           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.000006           1         5           total
```

Figure 3.2 – Number of write calls for ls, seq and a shell loop

3.3 Buffering decides the count

The same three-line `printf` program behaves differently depending on where `stdout` goes (Figure 3.3). Into a pipe the C library uses full buffering: **one** `write` of 80 bytes at exit. On a terminal (run under `script`, so `stdout` is a tty) it uses line buffering: **three** writes of 27, 20 and 33 bytes, one per newline. This is why the strace of Figure 2.2 shows one write to descriptor 1, while a person running `./syscalls` in a terminal sees each line as soon as it is printed.

```
dev1@dev1:~/os-lab9$ strace -e trace=write -o pipe.trace ./syscalls > out.txt ; cat pipe.trace
write(3, "hello, kernel\n", 14)         = 14
write(1, "open returned descriptor 3\nwrite"..., 80) = 80
+++ exited with 0 +++
dev1@dev1:~/os-lab9$ strace -e trace=write ./syscalls   (on the terminal)
write(1, "open returned descriptor 3\n", 27) = 27
write(3, "hello, kernel\n", 14)         = 14
write(1, "write sent 14 bytes\n", 20)   = 20
write(1, "read got 14 bytes: hello, kernel"..., 33) = 33
+++ exited with 0 +++
```

Figure 3.3 – The same program: one write into a pipe, three on a terminal

Answers in brief

Question	Answer
How many calls does `ls -l` make?	87 openat/read/write/close; 214 system calls in total
How does `ls` print?	One `write` on descriptor 1 (250 bytes)
Why is the first descriptor 3?	0, 1, 2 are stdin, stdout, stderr, inherited open; the kernel gives the lowest free one
Calls of `./syscalls`	41 total, 6 from `main`

CONCLUSIONS

A system call is visible and countable. strace showed that every `open`, `write`, `read` and `close` in my program became exactly one call into the kernel with the same arguments, and that the library translated `open` into `openat`. It also showed how much happens before `main`: 35 of the 41 calls of `syscalls` are the loader mapping the C library.

Descriptors are small integers that index the process's table of open files, and the kernel always hands out the lowest free one. The fact that my file got 3 is not a rule of the API: when I closed standard input before starting the program, the same `open` returned 0.

The count of `write` calls is decided by buffering. `ls` sends its whole listing in one write, `seq` sends 588895 bytes in 72 writes, and my three `printf` lines cost one write into a pipe but three on a terminal. A system call is expensive compared with a function call, so the C library batches output, and strace is the tool that makes that visible.